

■ Industrial Digital Twin

Production DevOps Project

A complete hands-on journey from code to production operations — using a virtual industrial plant as the learning environment.

Core idea

Students build and operate a software-based factory where virtual machines continuously generate temperature, pressure, flow, vibration and equipment-health telemetry. The platform is then containerized, automated, deployed to AWS/Kubernetes, secured, observed and deliberately tested under production failures.

Outcome: Build, deploy, observe, troubleshoot, secure and operate a distributed production platform.

1. Project Overview

The Industrial Digital Twin is a virtual representation of a real industrial plant. Instead of connecting to physical equipment, students create software-based machine simulators that behave like industrial assets and continuously produce telemetry.

The project intentionally evolves from a simple local application into a production-grade distributed platform. Each DevOps tool is introduced because the system has reached a problem that requires it.

High-level flow

Virtual Machines → MQTT → Kafka → Microservices → Database / Analytics → Monitoring → Alerts → Automation

Example plant assets

- Reactor — temperature, pressure, level and flow
- Pump — RPM, pressure, flow and vibration
- Compressor — temperature, pressure, RPM and vibration
- Cooling system — temperature, flow, fan speed and water level

2. Application Architecture

The platform is divided into independent services so students experience real microservice operations rather than a single monolithic application.

Digital Twin Platform ■■■ Machine Simulator Services ■■■ Telemetry Service ■■■ Alert Service ■■■
Maintenance Service ■■■ User Service ■■■ Reporting Service ■■■ Notification Service

Each service has its own source code, Dockerfile, CI pipeline, configuration, deployment, metrics and logs.

Event-driven data flow

Machine Simulator → MQTT Broker → Kafka → Consumer Services ■→ Storage ■→ Analytics ■→ Alerting

3. Complete DevOps Learning Journey

Stage	What students learn
01 — Linux	SSH, users, permissions, processes, systemd, networking, disk, CPU, memory and Linux logs.
02 — Git & GitHub	Branches, commits, pull requests, merge/rebase, tags, releases and repository strategy.
03 — Python & Bash	Machine simulators, health checks, deployment utilities, AWS automation and operational scripts.
04 — MQTT	Publishers, subscribers, topics, brokers and industrial telemetry ingestion.
05 — Kafka	Producers, consumers, topics, partitions, consumer groups, offsets, throughput and lag.
06 — Microservices	Independent services, APIs, service communication, configuration and failure boundaries.
07 — Docker	Dockerfiles, images, containers, networks, volumes, multi-stage builds and Docker Compose.
08 — CI/CD	Jenkins/GitHub Actions, webhooks, builds, tests, quality gates, image creation and artifact flow.
09 — DevSecOps	SonarQube, Trivy, Gitleaks, dependency scanning and IaC security with Checkov.
10 — AWS	VPC, subnets, IAM, ECR, EKS, RDS, load balancing, CloudWatch and related cloud services.
11 — Terraform	Modules, variables, outputs, state, remote backend, environment management and infrastructure automation.
12 — Ansible	Inventory, playbooks, roles, variables, templates, handlers and idempotent configuration.
13 — Kubernetes	Pods, Deployments, Services, Ingress, ConfigMaps, Secrets, HPA, PVC, RBAC and NetworkPolicies.
14 — Helm	Reusable charts, values, templates and environment-specific configuration.
15 — Argo CD / GitOps	Git-based deployment, synchronization, promotion, rollback, blue/green and canary patterns.
16 — Observability	Prometheus metrics, Grafana dashboards, application health and infrastructure visibility.
17 — Logging	Fluent Bit, Loki/OpenSearch, centralized logs and production troubleshooting.
18 — Distributed Tracing	OpenTelemetry, trace context and finding latency across service boundaries.
19 — Secrets	Vault/AWS Secrets Manager, Kubernetes integration, least privilege and rotation concepts.
20 — SRE	SLI, SLO, SLA, error budgets, incident response, RCA, runbooks and postmortems.
21 — Automation	Python, Boto3, Lambda, EventBridge and automated remediation.
22 — Disaster Recovery	Backups, restore, RTO, RPO, failover and recovery testing.

4. Production CI/CD Pipeline

Developer → GitHub → Jenkins / GitHub Actions → Build → Unit Tests → SonarQube → Security Scans → Docker Build → Trivy → ECR → GitOps Repository → Argo CD → EKS

Students learn why each stage exists and how a production change moves from source code to a controlled deployment.

5. Cloud & Infrastructure

Terraform provisions the AWS environment. The target architecture includes VPC, public/private subnets, EKS, RDS, Redis, ECR, load balancing, IAM and CloudWatch. Students also learn how to manage state and reusable Terraform modules.

6. Kubernetes Production Environment

EKS hosts the plant services. Students implement Deployments, Services, Ingress, ConfigMaps, Secrets, HPA, PVCs, StatefulSets where appropriate, RBAC, NetworkPolicies, resource requests/limits, probes and production deployment strategies.

7. GitOps Deployment

CI builds and publishes the image. The desired deployment state is stored in Git. Argo CD continuously reconciles that state into Kubernetes.

Code → CI → Image → GitOps Repository → Argo CD → EKS

8. Observability Stack

Metrics: Prometheus + Grafana Logs: Fluent Bit + Loki/OpenSearch Tracing: OpenTelemetry Alerting: Alertmanager

9. Real Production Incidents

The project becomes an SRE laboratory by intentionally breaking components and asking students to diagnose and recover them.

Reactor overheating

Temperature rises continuously. Students detect the anomaly, investigate telemetry and respond to the alert.

Kafka consumer failure

Consumers stop processing events. Kafka lag increases. Students identify the failing workload and restore processing.

Database outage

RDS becomes unavailable. API errors increase. Students investigate logs, metrics and dependencies, then execute recovery.

Memory leak

A service gradually consumes memory until pods are OOMKilled. Students correlate metrics, events and logs.

Bad deployment

A new version produces elevated 5xx responses. Students identify the regression and perform a rollback.

Traffic spike

Telemetry volume increases sharply. Students validate HPA behavior, Kafka throughput and application capacity.

10. Self-Healing & Automation

Failure → Prometheus → Alertmanager → Automation → Kubernetes / AWS → Recovery

- Restart unhealthy workloads
- Scale services during traffic spikes
- Trigger operational workflows
- Automate AWS resource checks with Python/Boto3
- Use Lambda/EventBridge for event-driven remediation

11. Security & Secrets

Security is integrated into the delivery lifecycle rather than added at the end.

- Gitleaks for secret detection
- SonarQube for code quality/security analysis
- Trivy for container and vulnerability scanning
- Checkov for Terraform/IaC security
- IAM and Kubernetes RBAC for least privilege
- Vault or AWS Secrets Manager for application secrets

12. Disaster Recovery

Students simulate production loss and practice backup, restore and recovery. They define RTO/RPO targets and test whether the system can actually recover.

13. Final Capstone Challenge

A final exercise simulates a major operational event: thousands of virtual machines generate a large telemetry burst while one or more services experience failures. Students must keep the platform available, identify bottlenecks, scale the system, respond to alerts, recover failed services and document the incident.

Target outcome: Build → Test → Secure → Containerize → Provision → Deploy → Observe → Troubleshoot → Scale → Recover → Automate

14. Complete Technology Coverage

Domain	Technology
OS & Scripting	Linux, Bash, Python
Source Control	Git, GitHub/GitLab
Containers	Docker, Docker Compose
CI/CD	Jenkins, GitHub Actions
Code Quality	SonarQube
Security	Trivy, Gitleaks, Checkov, dependency scanning
Cloud	AWS
Infrastructure as Code	Terraform
Configuration	Ansible
Orchestration	Kubernetes, EKS
Packaging	Helm
GitOps	Argo CD
Messaging	MQTT, Kafka
Data	PostgreSQL/RDS, Redis
Observability	Prometheus, Grafana
Logging	Fluent Bit, Loki/OpenSearch
Tracing	OpenTelemetry
Secrets	Vault / AWS Secrets Manager
Automation	Boto3, Lambda, EventBridge
SRE	SLI, SLO, SLA, error budgets, RCA, incident management
Reliability	Autoscaling, rollback, self-healing, backup/restore, DR

Final Student Portfolio Statement

“I built a production-grade Industrial Digital Twin platform where simulated industrial equipment generates real-time telemetry through MQTT and Kafka. I containerized microservices with Docker, provisioned AWS infrastructure using Terraform, configured supporting infrastructure with Ansible, deployed workloads on EKS using Helm and Argo CD, implemented CI/CD and DevSecOps, and built complete metrics, logging and distributed tracing using Prometheus, Grafana and OpenTelemetry. I also handled production incidents, autoscaling, self-healing and disaster recovery.”